3D Data Processing - Lab 2
Minimal Submission (Tasks 3 to 7 only)

Explanation and Build/Run Instructions

This document describes a deliberately minimal version of the lab submission. It implements only Tasks 3/7 through 7/7 in basic_sfm.cpp; Tasks 1/7 and 2/7 (in features_matcher.cpp) and the two OPTIONAL tasks are intentionally left as the original lab template, with their *Add your code here* placeholders intact and the surrounding *Code to be completed* comment blocks unchanged.

1. Scope of this Submission

The lab specification defines seven required tasks plus two OPTIONAL supplementary tasks. Tasks 1/7 and 2/7 live inside *features_matcher.cpp* (feature extraction + matching + geometric verification). Tasks 3/7 through 7/7 live inside *basic_sfm.cpp* (the incremental Structure from Motion pipeline). The two OPTIONAL tasks (alternative seed selection, alternative next-best-view) also live inside *basic_sfm.cpp*.

This submission implements **only Tasks 3/7, 4/7, 5/7, 6/7, and 7/7**. Tasks 1/7, 2/7, OPTIONAL 1, and OPTIONAL 2 are left as the original template. To make the submission still runnable, we ship the lab's pre-computed matcher outputs (*test_data1.txt* and *test_data2.txt*) inside the *datasets/* folder; these let *basic_sfm* reconstruct both provided datasets without needing the matcher.

> **Files modified:** only *src/basic_sfm.cpp*.
> **Files left as the original template:** *src/features_matcher.cpp*, *src/basic_sfm.h*, *src/features_matcher.h*, *src/io_utils.cpp*, *src/io_utils.h*, *src/matcher_app.cpp*, *src/sfm_app.cpp*, *CMakeLists.txt*, *README.txt*.

2. What is implemented and where

2.1 Task 5/7 - The ReprojectionError functor

> *Location: top of basic_sfm.cpp, struct ReprojectionError*

What it does. Defines the residual term used by Ceres in bundle adjustment: given a 6-DOF camera pose and a 3D point, it computes the 2D distance between the observed feature and the reprojection of that 3D point through the camera.

How it works. Templated *operator()* rotates the 3D point by the axis-angle rotation in *camera[0..2]* using *ceres::AngleAxisRotatePoint*, adds the translation in *camera[3..5]*, then divides by the third coordinate (canonical camera, K = identity, no distortion - because the matcher already produced normalised observations). The 2-D residual is (predicted - observed).

Why we did it this way. The *ceres::AutoDiffCostFunction* template parameters are *<Functor, residual_dim, camera_block_size, point_block_size>* = *<_, 2, 6, 3>*. Getting this order wrong is a classic source of silently wrong Jacobians; 6-then-3 matches the physical layout of the *parameters_* vector documented in *basic_sfm.h*.

2.2 Task 3/7 - Seed pair validation

> *Location: basic_sfm.cpp, incrementalReconstruction()*

What it does. Decides whether a candidate pair of cameras is good enough to bootstrap the entire reconstruction.

How it works. Three cascading checks:
(1) Fit both an Essential matrix and a Homography to the correspondences via RANSAC at threshold 0.001 (in normalised pixel units, i.e. ~1 pixel). Reject the pair if the homography explains as many or more matches than the Essential matrix - that means the scene is planar or the motion is pure rotation, both degenerate.
(2) Decompose E into the relative pose (R, t) using *cv::recoverPose*, which also enforces cheirality on the match inliers.
(3) Because *recoverPose* returns a unit translation vector, its components are direction cosines.

Reject the pair if neither |*tx*| nor |*ty*| reaches 0.5 (forward-dominant motion, which yields no triangulation baseline).

Why we did it this way. Each of the three checks targets a specific failure mode: H-vs-E catches degeneracy, recoverPose selects the geometrically consistent decomposition out of the four ambiguous ones, and the sideward test ensures the triangulation baseline is large enough to give 3D points meaningful precision. The 0.5 sideward threshold matches the lab spec's wording "mainly given by a sideward motion".

2.3 Task 4/7 - Triangulation

> *Location: basic_sfm.cpp, inside incrementalReconstruction inner loops*

What it does. After registering a new camera, this code triangulates each newly visible 3D point from the corresponding 2D observations in the new camera and a previously registered one.

How it works. Five steps:
(1) Build a 3x4 projection matrix [R | t] for each of the two cameras, by converting the axis-angle rotation block into a rotation matrix via *cv::Rodrigues* and concatenating with the translation block.
(2) Look up the canonical 2D observation of *pt_idx* in each camera (*obs0* in the new camera, *obs1* in the registered one).
(3) Run *cv::triangulatePoints*, which does linear DLT.
(4) De-homogenise the resulting 4-D point to 3-D Euclidean. Skip the point if the homogeneous w-coordinate is below 1e-8 in absolute value (point at infinity, would cause numerical blow-up).
(5) Cheirality check: compute the depth of the new 3-D point in *each* of the two camera frames; accept only if both are positive. The spec literally says "In case of success (cheirality constraint satisfied) execute the following instructions" - we follow that exactly, with no additional rejection criteria.

Why we did it this way. The *w-guard* is a small but important defence against undefined behaviour from *cv::triangulatePoints* when a point lies at or near infinity; without it, a single match on a far-away background object can produce a coordinate of magnitude 10^9 that wrecks downstream BA. The two-sided cheirality check rejects geometrically valid but numerically poor triangulations.

2.4 Task 6/7 - Residual block in bundle adjustment

> *Location: basic_sfm.cpp, bundleAdjustmentIter()*

What it does. For every observation owned by a currently-registered (camera, point) pair, adds a corresponding residual block to the Ceres optimisation problem.

How it works. Three components per residual block:
(1) The cost function: built via *ReprojectionError::Create* with the observed 2-D coordinates as constants.
(2) The robust loss: *ceres::CauchyLoss(2 * max_reproj_err_)*. Cauchy aggressively suppresses outlier residuals - much more so than Huber - which is what we want given that the matcher's RANSAC stage is not perfect and a few outliers always slip through.
(3) The parameter pointers: *camera_block* at offset *camera_block_size_ * cam_pose_index_[i_obs]* from *parameters_.data()*; *point_block* at offset *camera_block_size_ * num_cam_poses_ + point_block_size_ * point_index_[i_obs]*.

Why we did it this way. Cauchy was chosen after a separate experiment (in our full submission) that compared NULL, Huber, and Cauchy on the same datasets; on average Cauchy gave the

best balance of point density and reprojection error. The scale parameter *2 * max_reproj_err_* is the value suggested in the lab specification.

2.5 Task 7/7 - Divergence detection

> *Location: basic_sfm.cpp, end of incrementalReconstruction main loop*

What it does. Detects when the reconstruction has gone wrong (e.g. the BA jumped to a non-physical local minimum, or the outlier filter pruned almost everything) and aborts so the seed search loop can try a different starting pair.

How it works. Two checks:
(1) **Catastrophic point loss.** If fewer than 10 points survived the outlier-rejection pass already in place, the reconstruction has collapsed; return false.
(2) **Camera position blow-up.** Snapshot the parameter vector before BA and compare it after. If the average translation displacement of cameras that were already registered before this iteration exceeds the scene's bounding-box diagonal (computed as *max_dist / 5*, with a floor of 2.0 normalised units), the optimisation has jumped to a bad local minimum; return false.

Why we did it this way. The point-count check alone misses divergences where the geometry blows up but the outlier filter happens to spare some points. The camera-displacement check fills that gap. The *max_dist / 5* threshold is scene-relative (*max_dist* is already scene-relative, being 5x the bounding-box diagonal computed earlier in the same loop), so it adapts automatically as the reconstruction grows.

3. What is intentionally NOT implemented

The following sections of the template are left exactly as the lab shipped them, with their *Code to be completed* comment block intact and an *Add your code here* placeholder still inside:

Task 1/7 (geometric verification of matches in *features_matcher.cpp*, line 133).
Task 2/7 (modern features extraction and matching in *features_matcher.cpp*, lines 39 and 77).
OPTIONAL 1 (alternative seed selection in *basic_sfm.cpp*, line 480).
OPTIONAL 2 (alternative next-best-view in *basic_sfm.cpp*, line 650).

Because tasks 1/7 and 2/7 are not implemented, the **matcher** executable will compile and run, but the matcher cannot produce useful output - all of its post-detection logic is a placeholder. The **basic_sfm** executable, in contrast, is fully functional given a valid input file, since basic_sfm only reads the .txt produced by the matcher and never needs to call the matcher's internal code.

> **How we work around this:** the lab ships pre-computed matcher output files (*test_data1.txt* and *test_data2.txt*) inside *datasets/*, produced by the course's reference matcher. Running *./bin/basic_sfm datasets/test_data1.txt cloud1.ply* uses these pre-computed files and reconstructs the dataset using exactly our tasks 3 through 7. We have included these files in this submission package.

4. Build Instructions

From the project root (the folder containing *CMakeLists.txt*):

```
mkdir -p build
cd build
cmake -DCMAKE_BUILD_TYPE=Release ..
make -j$(nproc)
```

```
cd ..
```

After a successful build you should see two executables under *./bin/*:

```
./bin/matcher
./bin/basic_sfm
```

Required dependencies (Ubuntu 22.04+ packages):
build-essential, cmake, libboost-filesystem-dev, libopencv-dev, libomp-dev, libceres-dev, libeigen3-dev.

5. Run Instructions

5.1 Reconstruct Dataset 1 (9 images)

```
./bin/basic_sfm  datasets/test_data1.txt  cloud1.ply
```

5.2 Reconstruct Dataset 2 (15 images)

```
./bin/basic_sfm  datasets/test_data2.txt  cloud2.ply
```

Each run prints progress to standard output, including the seed pair selected, each newly registered camera, and the final summary. On success the last line is *Recostruction completed, exiting* (the typo is in the original lab template) and the .ply file is written to disk. Open the .ply in MeshLab to inspect the result.

5.3 Sample expected output

```
$ ./bin/basic_sfm datasets/test_data1.txt cloud1.ply
Header: 9 4106 16193
Seed pair (0, 1): inliers E=2010 H=331
camera[0] r_vec=(0,0,0) t_vec=(0,0,0)
camera[1] r_vec=(...) t_vec=(...)
ADDED 1100+ new points
...
Using 9 over 9 cameras
Recostruction completed, exiting
```

Successful reconstruction is indicated by all cameras being registered (the *Using N over N cameras* line) and the final *Recostruction completed, exiting* message. The output .ply should contain a few thousand 3D points plus N green dots representing the camera centres.

5.4 Note on the matcher executable

> The matcher executable also builds, but because tasks 1/7 and 2/7 are intentionally left unimplemented in this submission, running the matcher would not produce a usable output file. **Use the shipped test_data*.txt files instead**; they are exactly the format basic_sfm expects.

If you want to run the matcher on the original images, you need the full submission (with tasks 1/7 and 2/7 implemented). The minimal submission was prepared specifically to demonstrate tasks 3 through 7 in isolation.

6. Pre-built Verification

The folder *results/* contains **cloud1.ply** and **cloud2.ply**, the output we obtained when running the steps in section 5 in our build environment. They are not part of the deliverable and can be deleted; they are included only so you can compare against your own runs to confirm everything is wired up correctly.

If your re-run produces clouds with the same camera count (9 / 9 for DS1, 15 / 15 for DS2) and a similar number of points (~3000 for DS1, ~3700 for DS2), the implementation is working correctly. Exact point counts vary slightly due to the stochastic RANSAC-style outlier rejection inside the BA-redo loop.

7. File-by-File Status (versus lab template)

Quick reference summary of what each file contains.

CMakeLists.txt: unchanged from template

README.txt: unchanged from template

src/io_utils.h, .cpp: unchanged from template

src/matcher_app.cpp: unchanged from template (driver for matcher)

src/sfm_app.cpp: unchanged from template (driver for basic_sfm)

src/basic_sfm.h: unchanged from template (header)

src/features_matcher.h: unchanged from template (header)

src/features_matcher.cpp: UNCHANGED. Tasks 1/7 and 2/7 are NOT implemented; the original 'Code to be completed' blocks and 'Add your code here' placeholders remain intact.

src/basic_sfm.cpp: MODIFIED. Tasks 3/7, 4/7, 5/7, 6/7, 7/7 are implemented in their respective marked sections. OPTIONAL 1 and OPTIONAL 2 are NOT implemented; their original template blocks remain intact.

datasets/3dp_cam.yml: unchanged from template (camera calibration)

datasets/test_data1.txt: shipped pre-computed matcher output for DS1

datasets/test_data2.txt: shipped pre-computed matcher output for DS2

results/cloud1.ply: our pre-built output, for verification only

results/cloud2.ply: our pre-built output, for verification only

EXPLANATION.pdf: this document

8. Lab-spec Compliance Audit

Direct mapping of each implemented task to the literal text of the lab specification (Lab2 - Structure from Motion.pdf). For each task, the left column quotes the spec and the right column states what our implementation does.

Task 3/7

Spec wording	Our implementation
"Extract the essential matrix E and the homography matrix H"	<i>cv::findEssentialMat</i> + <i>cv::findHomography</i> at threshold 0.001
"check the number of inliers for both models"	Compare <i>countNonZero(inlier_mask_E)</i> vs <i>countNonZero(inlier_
"the number of inliers for E should be greater than the number of inliers for H"	If <i>inliers_E <= inliers_H</i>, return false.
"recover from E the initial rigid-body transformation"	<i>cv::recoverPose(E, ..., init_r_mat, init_t_vec, inlier_mask_E)</i>.
"check whether the recovered transformation is primarily sideward	Reject if both <i>|tx| < 0.5</i> and <i>|ty| < 0.5</i> (forward-dominant m
"promote the pair as a seed pair, setting the estimated	Pair is implicitly promoted (function returns true) and init_r_mat / init_t_vec a

Task 4/7

Spec wording	Our implementation
"Triangulate a set of newly registered 3D points from a pair of observations"	Per-point loop iterating over candidates visible in both cameras.
"Use the OpenCV cv::triangulatePoints() function"	Direct call with the two 3x4 projection matrices and the 2D observations.
"remembering to check the cheirality constraint for both cameras"	Compute depth in both camera frames; accept only when both are positive
"In case of success (cheirality constraint satisfied) execute the following instructions"	Inside the cheirality-positive branch we run exactly the four lines listed in the

Task 5/7

Spec wording	Our implementation
"Implement an auto-differentiable cost function for the Ceres Solver problem"	<i>struct ReprojectionError</i> with a templated <i>operator()</i> and a sta
"To rotate a point given an axis-angle rotation, use the Ceres function AngleAxisRotatePoint()"	First line of <i>operator()</i> calls <i>ceres::AngleAxisRotatePoint(camera,
"WARNING: When dealing with the AutoDiffCostFunction template parameters, pay attention to the order"	Order is <i><Functor, residual_dim=2, camera_block=6, point_block=3&g
"we are dealing with a normalized, canonical camera"	No K, no distortion. Project by simple division by depth: <i>predicted_x = p[

Task 6/7

Spec wording	Our implementation
"add a residual block inside a Ceres Solver problem"	<i>problem.AddResidualBlock(cost_function, loss_function, camera_block,
"The camera position blocks have size (camera_block_size) of 6 elements, while the point position blocks have size (point_block_	<i>camera_block = parameters_.data() + cam_idx * camera_block_size* (point_bloc
"experiment with different robust loss kernels: ceres::HuberLoss(a), ceres::CauchyLoss(a)... where 'a' is a scale parameter	<i>new ceres::CauchyLoss(2.0 * max_reproj_err)</i>. The Cauchy kernel...

Task 7/7

Spec wording	Our implementation
"Check at each registration iteration if the reconstruction has diverged"	Two checks after every BA iteration: (a) fewer than 10 valid points rema
"In this case, just reset the reconstruction and start from scratch with a new pair of seeds"	Either trigger returns false from <i>incrementalReconstruction</i>, which th

File-by-file integrity (vs. lab template)

File	Lines differing
CMakeLists.txt	0
README.txt	0
src/io_utils.{h,cpp}	0
src/matcher_app.cpp	0 (do-not-modify file, per spec)
src/sfm_app.cpp	0 (do-not-modify file, per spec)
src/basic_sfm.h	0
src/features_matcher.h	0
src/features_matcher.cpp	0 (tasks 1/7 and 2/7 left as template)
src/basic_sfm.cpp - OPTIONAL 1 section	0 (template intact)
src/basic_sfm.cpp - OPTIONAL 2 section	0 (template intact)
src/basic_sfm.cpp - tasks 3 to 7 sections	modified